Modelling & Simulation

Solutions to Exercise Series No. 01

University 08 Mai 1945 – Guelma · Computer Science Dept. · 4th-year Engineering (AI)

Detailed solutions with explanations (English, with Arabic summaries)

0. Key concepts and vocabulary

Before solving, here are the three classification criteria from the course (Chapter I, “Topologie / classification des systèmes”). A quick test question is given for each one.

Criterion	How to decide	Arabic / ملاحظة
Continuous vs Discrete	Continuous: the state changes at every instant and is described by real-valued variables and differential equations (water level, temperature). Discrete: the state changes only at precise instants / steps and takes countable values (number of clients, step k = 0, 1, 2…).	مستمر (Continu) = متقطع لحظة؛ كل في يتغير قابلة وبقيم محددة لحظات عند يتغير (Discret) = للعد.
Deterministic vs Stochastic	Deterministic: same inputs and same parameters give always the same output (no randomness). Stochastic: at least one random variable (probability law) → two runs give different results.	حتمي (Déterministe) = عشوائي عشوائية؛ لا عشوائية متغيرات توجد (Stochastique) = احتمالية بقوانين.
Static vs Dynamic	Static: time plays no role (a snapshot, an equilibrium, a one-shot calculation). Dynamic: the state evolves with time (transitions, evolution step by step or continuously).	ثابت (Statique) = ديناميكي مؤثر؛ غير الزمن (Dynamique) = الزمن مع تتغير الحالة.

State variable = the minimal set of variables that describes the system at a given moment, so that, knowing it (and the inputs), we can predict the future. (الحالة متغير: المتغيرات التي تصف النظام في لحظة معينة.)

Quick tip: ask three questions in order: (1) Does time matter? (2) Is there any randomness? (3) Does the state change continuously or at steps/events?

Exercise 01 – Classification of six situations

Questions 1 & 2 – Classification and justification

Summary table (Question 1):

Situation	Continuous / Discrete	Deterministic / Stochastic	Static / Dynamic
S1 – Processor cooling	Continuous	Deterministic	Dynamic
S2 – Queue of requests	Discrete	Stochastic	Dynamic
S3 – Network cost	Discrete (integer quantities)	Deterministic	Static
S4 – Bacteria population	Discrete (hourly steps)	Deterministic	Dynamic
S5 – Sensor network	Discrete	Stochastic	Dynamic

Situation	Continuous / Discrete	Deterministic / Stochastic	Static / Dynamic
S6 – Robot trajectory	Continuous	Deterministic	Dynamic

Justifications (Question 2) – each answer is tied to a sentence of the statement.

Situation 1 – Processor cooling → Continuous, Deterministic, Dynamic

- **Dynamic:** the temperature "decreases progressively" after the program stops, so it evolves with time.
- **Continuous:** the evolution is represented by a **differential equation**; temperature is a real number defined at every instant t.
- **Deterministic:** "parameters are known and constant", and there is no random term. Given the initial temperature, the curve is unique.

A typical model is Newton's law of cooling:

$$dT(t)/dt = -k \cdot (T(t) - T_amb) \quad \rightarrow \quad T(t) = T_amb + (T_0 - T_amb) \cdot e^{(-k \cdot t)}$$

ملخص بالعربية
المعالج يبرد مع الزمن بشكل مستمر، والمعادلة التفاضلية تصف درجة الحرارة في كل لحظة.
البارامترات معروفة وثابتة ولا توجد عشوائية، إذن: مستمر + حتمي + ديناميكي.

Situation 2 – Queue of requests → Discrete, Stochastic, Dynamic

- **Dynamic:** "the number of waiting requests evolves over time".
- **Stochastic:** arrival instants are random and processing times are random (statement says it twice).
- **Discrete:** the number of requests is an integer (0, 1, 2, …) and it changes **only at events**: an arrival (+1) or a departure (−1). Between two events nothing changes. This is a typical discrete-event system.

ملخص بالعربية
الطلبات تصل في أوقات عشوائية وزمن المعالجة عشوائي، فالنظام عشوائي.
عدد الطلبات عدد صحيح يتغير فقط عند حدث (وصول أو مغادرة)، إذن: متقطع + عشوائي + ديناميكي.

Situation 3 – Network cost → Discrete, Deterministic, Static

- **Static:** "without studying its temporal evolution"; the calculation concerns one given configuration (a snapshot).
- **Deterministic:** unit prices are known; same configuration → same cost, no randomness.
- **Discrete:** the quantities (routers, switches, cables) are integer counts. Note: for a static model the continuous/discrete criterion is less important (there is no evolution); the natural answer is "discrete" because the variables are countable.

$$Cost = n_routers \cdot p_router + n_switches \cdot p_switch + n_cables \cdot p_cable$$

ملخص بالعربية
الحساب يخص تشكيلة واحدة فقط دون دراسة تطور زمني، فالنموذج ثابت (ستاتيكي).
الأسعار معروفة فلا عشوائية، والكميات أعداد صحيحة: متقطع + حتمي + ثابت.

Situation 4 – Bacteria population → Discrete, Deterministic, Dynamic

- **Dynamic:** the population evolves from hour to hour.

- **Discrete:** time advances by **steps** (every hour); the number at hour k is computed from hour k−1 (a recurrence relation), not from a differential equation.
- **Deterministic:** "the growth rate is known and the results are predictable".

$$N(k+1) = (1 + r) \cdot N(k) \quad with\ r = growth\ rate\ per\ hour$$

ملخص بالعربية
الزمن يُحسب بالساعات (خطوات منفصلة) والعدد يُحسب من الساعة السابقة بعلاقة تراجعية.
معدل النمو معروف والنتائج متوقعة: متقطع + حتمي + ديناميكي.

Situation 5 – Sensor network → Discrete, Stochastic, Dynamic

- **Stochastic:** each sensor fails with a **probability** (random behaviour).
- **Dynamic:** the number of working sensors is observed **every minute**, so the state is followed over time (sensors can fail as time goes by).
- **Discrete:** each sensor has two states (works / failed); the number of working sensors is an integer between 0 and 100; observation happens at discrete instants (every minute).

ملخص بالعربية
كل حساس يعمل أو يتعطل باحتمال معين (عشوائي)، والعدد يُلاحَظ كل دقيقة (ديناميكي).
عدد الحساسات العاملة عدد صحيح بين 0 و100: متقطع + عشوائي + ديناميكي.

Situation 6 – Robot trajectory → Continuous, Deterministic, Dynamic

- **Dynamic:** position and velocity "evolve continuously over time".
- **Continuous:** position x(t) and velocity v(t) are real-valued functions of continuous time, governed by differential equations (Newton's second law).
- **Deterministic:** "forces and initial conditions are known and no random disturbance is considered".

$$dx/dt = v, \quad m \cdot dv/dt = F(t) \quad with\ x(0) = x_0,\ v(0) = v_0$$

ملخص بالعربية
الموضع والسرعة يتغيران باستمرار مع الزمن وتحكمهما معادلات تفاضلية (قانون نيوتن).
القوى والشروط الابتدائية معروفة ولا يوجد اضطراب عشوائي: مستمر + حتمي + ديناميكي.

Question 3 – State variables

The state variables are the quantities that evolve and describe the system at each moment. Parameters (constants such as k, r, prices) are **not** state variables.

Situation	State variable(s)	Remarks (inputs / parameters)
S1	T(t): processor temperature	Parameters: k (cooling coefficient), T_amb, initial temperature T_0
S2	N(t): number of requests waiting (or in the system); optionally the server state (busy / idle)	Random inputs: inter-arrival times and service times
S3	No evolving state (static). Decision variables: n_routers, n_switches, n_cables. Output: total cost C	Parameters: unit prices
S4	N(k): number of bacteria at hour k	Parameter: growth rate r; initial population N(0)

Situation	State variable(s)	Remarks (inputs / parameters)
S5	X(k): number of operational sensors at minute k (0…100). Finer version: the vector of states of the 100 sensors, each 0 or 1	Parameter: failure probability p of each sensor
S6	Position x(t) and velocity v(t) (a vector in 2D/3D)	Inputs: applied forces F(t); initial conditions x_0 , v_0 ; parameter m (mass)

ملخص بالعربية
متغيرات الحالة هي الكميات التي تتغير وتصف النظام في كل لحظة (درجة الحرارة، عدد الطلبات، عدد البكتيريا، عدد الحساسات العاملة، الموضع والسرعة).
أما الثوابت مثل k وr والأسعار فهي بارامترات وليست متغيرات حالة.

Question 4 – Why are the models of Situations 1 and 6 continuous?

- The state variables (temperature T; position x and velocity v) can take **any real value** in an interval, not only integers.
- Time is a continuous variable $t \in \mathbb{R}^+$: the state is defined and changes at **every instant**, with no “jumps” at particular moments.
- The evolution is expressed by **differential equations** (dT/dt, dx/dt, dv/dt) – the rate of change is given at each instant.
- On a computer such models are simulated by numerical integration with a small time step (Euler, Runge-Kutta) – this is “continuous simulation” (Chapter III).

ملخص بالعربية
المتغيرات تأخذ أي قيمة حقيقية، والزمن مستمر، والتطور يوصف بمعادلات تفاضلية، لذلك النموذج مستمر.

Question 5 – Role of randomness in Situations 2 and 5

Situation 2 (queue)

- Two sources of randomness: **inter-arrival times** (e.g. Poisson arrivals, exponential gaps) and **service times** (e.g. exponential law).
- Because of them, the queue length is unpredictable: two runs with the same parameters give two different trajectories. Even if the average service capacity is larger than the average arrival rate, **random bursts** of arrivals create waiting.
- Consequence: we cannot give one exact value; we estimate **averages** (mean waiting time, mean queue length) and probabilities, using probability theory or many simulation replications.

Situation 5 (sensors)

- The failure of each sensor is a random event with probability p. The number of failed sensors is a random variable (for independent sensors it follows a **Binomial law** B(100, p)).
- Expected number of working sensors = 100 · (1 − p), but the real value observed each minute fluctuates around this mean.
- Consequence: we study the **distribution** (mean, variance, probability that fewer than a given number of sensors work) instead of a single trajectory.

ملخص بالعربية
العشوائية هي مصدر عدم اليقين: في الطابور (أزمنة الوصول وأزمنة الخدمة) وفي الحساسات (احتمال العطل).

بسببها نحصل على نتائج مختلفة في كل تشغيل، لذلك نحسب المتوسطات والاحتمالات ونكرر المحاكاة عدة مرات.

Question 6 – From a deterministic to a stochastic model (Situation 4)

Current deterministic model: **N(k+1) = (1 + r)·N(k)**. Several modifications introduce randomness:

1. **Random growth rate:** replace r by a random variable R(k) (e.g. R(k) ~ Normal(r, σ^2) or uniform on [r_1 , r_2]): N(k+1) = (1 + R(k))·N(k). The environment (temperature, food) varies randomly from hour to hour.
2. **Individual-level model (birth–death process):** each bacterium, independently, divides with probability p and dies with probability q during one hour. Then N(k+1) = N(k) + B(k) − D(k), where B(k) ~ Binomial(N(k), p) and D(k) ~ Binomial(N(k), q). The mean growth is E[N(k+1)] = (1 + p − q)·E[N(k)], so choose p − q = r to keep the same average behaviour.
3. **Additive noise:** N(k+1) = (1 + r)·N(k) + ε(k), with ε(k) a random perturbation (noise).

After this change the model becomes stochastic: we must **run the simulation many times** (replications) and compute the mean and a confidence interval of N(k).

ملخص بالعربية
لتحويل النموذج الحتمي إلى عشوائي نُدخل عنصراً عشوائياً: إما نجعل معدل النمو r متغيراً عشوائياً R(k)،
أو نفترض أن كل بكتيريا تنقسم باحتمال p وتموت باحتمال q في كل ساعة. بعدها نكرر المحاكاة عدة مرات ونحسب المتوسط.

Exercise 02 – Single-teller bank branch

Statement recap: one teller; clients arrive one after another; if the teller is busy, they wait their turn. We want to study how the branch works.

A. Which questions can we study?

- What is the **average waiting time** of a client before being served? And the **maximum** waiting time?
- What is the **average number of clients waiting** (queue length), and the maximum length (size of the waiting room needed)?
- What is the **utilisation rate** of the teller (percentage of time busy) and the percentage of idle time?
- What is the **probability that a client waits more than T minutes** (e.g. 10 min)?
- How many clients are served per hour (throughput)?
- **What-if scenarios:** what happens if arrivals increase by 20 %? If the service gets faster? If a **second teller** is opened? If clients leave when the queue is too long?

ملخص بالعربية
الأسئلة الممكنة: متوسط زمن الانتظار، طول الطابور، نسبة انشغال الشباك، احتمال أن ينتظر الزبون أكثر من مدة معينة، وسيناريوهات مثل إضافة شباك ثان أو زيادة عدد الزبائن.

B. Identification of the elements of the system

Element	Answer for the bank	بالعربية
System studied	The bank branch seen as a service system: waiting line + teller (the boundary: from the	النظام: البنكية الوكالة (الطابور + الشباك)

Element	Answer for the bank	بالعربية
	moment a client arrives until he leaves after service).	
Entities (circulate)	The clients (customers). Each client has attributes: arrival time, service duration, start-of-service time, departure time.	الزبائن :الكيانات
Resources	The teller (capacity = 1) – a client can be served only if it is free. Also the waiting area / queue (assumed unlimited here).	وقاعة (1 = سعة) الشباك :الموارد الانتظار
Variables	Inputs (random): inter-arrival times A(i), service times S(i). Parameters: arrival rate λ, service rate μ, number of tellers c = 1, queue discipline FIFO. State variables: N(t) = clients in the system, Q(t) = clients waiting, teller state (busy / idle), and the simulation clock. Outputs: waiting time Wq(i), time in the system W(i).	المتغيرات :المدخلات (أزمنة الوصول والخدمة)، البارامترات (λ، μ)، متغيرات الحالة (N, Q, حالة الشباك)
Performance indicators	Mean waiting time Wq; mean time in system W; mean queue length Lq; mean number in system L; teller utilisation ρ; throughput; P(wait > T); maximum queue length.	مؤشرات الأداء: متوسط الانتظار، طول الطابور، معدل الاستعمال...

C. Simplified representation (model) of the branch

We build a **conceptual model** (Chapter I methodology: define the problem → delimit the system → identify variables → formalise). This is the classical **M/M/1 queue** (Kendall notation):

- **Arrivals:** clients arrive one by one, with exponentially distributed inter-arrival times (equivalently a Poisson process) of rate λ.
- **Service:** one teller; service duration exponentially distributed with rate μ (mean service time 1/μ).
- **Rules:** a client who arrives when the teller is free is served immediately; otherwise he joins the queue. The discipline is **FIFO** (first in, first out). The queue has unlimited capacity, no client leaves before being served, the teller never stops working.
- **Stability condition:** ρ = λ/μ < 1 (otherwise the queue grows forever).

Graphic view:

$$Arrivals\ (\lambda) \rightarrow [\ Queue\ \circ\circ\circ\circ\ ] \rightarrow (\ Teller\ \mu\ ) \rightarrow Departures$$

State of the system: N(t) ∈ {0, 1, 2, …}. Transitions: arrival N → N+1 (rate λ); departure N → N−1 (rate μ, only if N ≥ 1). It is a discrete-event, stochastic, dynamic system (a birth–death Markov chain).

Analytical results for M/M/1 (mathematical method – Chapter II):

Indicator	Formula	Example λ = 0.2 / min, μ = 0.25 / min
Utilisation ρ	$\rho = \lambda / \mu$	0.2 / 0.25 = 0.8 (80 %)
Mean number in system L	$L = \rho / (1 - \rho)$	0.8 / 0.2 = 4 clients
Mean queue length Lq	$Lq = \rho^2 / (1 - \rho)$	0.64 / 0.2 = 3.2 clients
Mean time in system W	$W = 1 / (\mu - \lambda)$	1 / 0.05 = 20 min

Indicator	Formula	Example λ = 0.2 / min, μ = 0.25 / min
Mean waiting time Wq	$Wq = \rho / (\mu - \lambda)$	0.8 / 0.05 = 16 min
P(teller idle)	$P_0 = 1 - \rho$	20 %

Check with Little's law: $Lq = \lambda \cdot Wq = 0.2 \times 16 = 3.2$ ✓ and $L = \lambda \cdot W = 0.2 \times 20 = 4$ ✓.

ملخص بالعربية
النموذج المبسط هو طابور M/M/1: زبائن يصلون بمعدل λ، شباك واحد بمعدل خدمة μ، وقاعدة الخدمة: الأول وصولاً هو الأول خدمةً (FIFO).
شرط الاستقرار: $\rho = \lambda/\mu$ أقل من 1. في المثال: ρ = 0.8، متوسط الانتظار 16 دقيقة، ومتوسط طول الطابور 3.2 زبون.

D. Can we simulate the operation? – Yes

Yes. The system is a **discrete-event system**, so we can simulate it by **discrete-event simulation** (Chapter III). We generate random inter-arrival and service times (pseudo-random number generators), then follow the system event by event.

Simulation principle

4. **State:** teller busy / idle, queue length, clock. **Event list:** next arrival, next departure.
5. **Initialisation:** clock = 0, empty queue, teller idle, schedule the first arrival.
6. **Loop:** take the earliest event and advance the clock to its time. If it is an **arrival** → schedule the next arrival; if the teller is idle, start service and schedule the departure, otherwise add the client to the queue. If it is a **departure** → if the queue is not empty, take the first client and start his service (schedule his departure), otherwise the teller becomes idle.
7. **Statistics:** record each client's waiting time, the busy time of the teller, the queue length over time.
8. **Stop** after a given number of clients or a given time; compute the indicators; **repeat** (replications) and give averages with confidence intervals.

Hand simulation example (5 clients)

Suppose the random generator gave the inter-arrival times 2, 3, 1, 4, 2 (min) and the service times 3, 2, 4, 1, 3 (min). Rule: start = max(arrival, time the teller becomes free); wait = start − arrival; departure = start + service.

Client	Arrival	Service start	Service time	Departure	Waiting time	Teller free at
1	2	2	3	5	0	5
2	5	5	2	7	0	7
3	6	7	4	11	1	11
4	10	11	1	12	1	12
5	12	12	3	15	0	15

- Mean waiting time = (0 + 0 + 1 + 1 + 0) / 5 = **0.4 min**; 2 of 5 clients had to wait.
- Teller busy time = 3 + 2 + 4 + 1 + 3 = 13 min out of 15 → utilisation = 13/15 ≈ **86.7 %** (idle from t = 0 to 2).

- Maximum queue length = **1** client (at t = 6 and t = 10).

Such a short run is only an illustration; it is far too small to estimate real averages.

Python implementation

The code below uses the recurrence start = max(arrival, teller free time) and simulates 100 000 clients with λ = 0.2 and μ = 0.25.

```
import random

def simulate(lam=0.2, mu=0.25, n=100000, seed=42):
    random.seed(seed)
    t_arr = 0.0        # arrival time of the current client
    t_free = 0.0       # time at which the teller becomes free
    total_wait = busy = 0.0
    waited = 0         # clients who waited more than 10 min
    for _ in range(n):
        t_arr += random.expovariate(lam)    # next inter-arrival time
        start = max(t_arr, t_free)          # service starts when teller free
        wait = start - t_arr
        service = random.expovariate(mu)    # service duration
        t_free = start + service            # departure time
        total_wait += wait
        busy += service
        if wait > 10:
            waited += 1
    return total_wait / n, busy / t_free, waited / n

wq, util, p10 = simulate()
print(f'Mean wait = {wq:.2f} min | Utilisation = {util:.2%} | P(wait>10) = {p10:.2%}')
```

Result obtained when running this code: mean wait ≈ 16.74 min, utilisation ≈ 80.4 %, P(wait > 10 min) ≈ 49.5 %. These values are close to the theory (Wq = 16 min, ρ = 80 %, P(Wq > 10) = ρ·e^(−(μ−λ)·10) ≈ 48.5 %). The small difference is normal: simulation gives an **estimate** with statistical error, which decreases when the number of clients or the number of replications increases.

What the simulation brings beyond the formula: we can easily change the assumptions (non-exponential service times, two tellers, impatient clients, limited waiting room) where no simple formula exists.

ملخص بالعربية

نعم، يمكن محاكاة النظام لأنه نظام متقطع الأحداث: نولّد أزمنة الوصول والخدمة عشوائياً ونتابع الأحداث (وصول / مغادرة) واحداً تلو الآخر.

في مثال يدوي بـ 5 زبائن: متوسط الانتظار 0.4 دقيقة ونسبة انشغال الشباك 86.7%. وفي محاكاة 100000 زبون حصلنا على متوسط انتظار ≈ 16.7 دقيقة وهو قريب من القيمة النظرية 16 دقيقة.

ميزة المحاكاة: نغيّر الفرضيات بسهولة (شباكان، زبائن ينسحبون، قاعة انتظار محدودة) حتى عندما لا توجد صيغة رياضية.

Final summary

Situation	Type	State variable	Randomness?
S1	Continuous · Dynamic	T(t)	No
S2	Discrete · Dynamic	N(t) requests waiting	Yes: arrivals, service
S3	Discrete · Static	None (counts → cost)	No

Situation	Type	State variable	Randomness?
S4	Discrete · Dynamic	N(k) bacteria	No (can be added)
S5	Discrete · Dynamic	X(k) working sensors	Yes: failures
S6	Continuous · Dynamic	x(t), v(t)	No

Exam tip: always justify with a sentence taken from the statement (e.g. “parameters are known” → deterministic; “differential equation” → continuous; “without studying temporal evolution” → static).